



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA EN
SISTEMAS DE TELECOMUNICACIONES

TRABAJO FIN DE GRADO

Diseño e Implementación de una Herramienta de
Gemelo Digital (Digital Twin) para la Planificación
y Visualización de Cobertura en Redes 5G
Tácticas Desplegables

Autor: Álvaro Martínez Téllez

Tutor: Miguel Ángel Ortuño Pérez

Curso académico 2025-2026



Este trabajo se distribuye bajo los términos de la licencia internacional CC BY-SA International License (Creative Commons Attribution-ShareAlike 4.0). Usted es libre de *(a) compartir*: copiar y redistribuir el material en cualquier medio o formato para cualquier propósito, incluso comercialmente; y *(b) adaptar*: remezclar, transformar y construir a partir del material para cualquier propósito, incluso comercialmente. La licenciente no puede revocar estas libertades en tanto usted siga los términos de la licencia:

- *Atribución.* Usted debe dar crédito de manera adecuada, brindar un enlace a la licencia, e indicar si se han realizado cambios. Puede hacerlo en cualquier forma razonable, pero no de forma tal que sugiera que usted o su uso tienen el apoyo de la licenciente.
- *Compartir igual.* Si remezcla, transforma o crea a partir del material, debe distribuir su contribución bajo la misma licencia del original.

No hay restricciones adicionales — No puede aplicar términos legales ni medidas tecnológicas que restrinjan legalmente a otras a hacer cualquier uso permitido por la licencia.

Agradecimientos

Reconocimiento a tutor, familia, compañeros y universidad.

A quienes me apoyaron durante este camino.

Resumen

Síntesis ejecutiva del proyecto en español (300 palabras): problema, solución, resultados clave.

Abstract

Versión en inglés del resumen para publicaciones internacionales.

Acrónimos

Términos técnicos definidos (5G, RSRP, EIRP, API, etc.)

Índice general

1. Introducción	1
1.1. Contexto y Motivación	1
1.2. Descripción del Problema	1
1.3. Objetivos del Trabajo	1
1.4. Metodología y Planificación	1
1.5. Estructura del Documento	1
2. Estado del Arte	2
2.1. Evolución de las redes 5G NR	2
2.2. Redes tácticas y burbujas 5G desplegables	3
2.3. Modelos de propagación radioeléctrica	3
2.3.1. Pérdidas en el espacio libre (FSPL)	4
2.3.2. Modelos Okumura-Hata y COST-231	4
2.3.3. Modelo Longley-Rice (ITM)	4
2.3.4. 3GPP TR 38.901	5
2.3.5. Comparativa de modelos	5
2.4. Herramientas de planificación radioeléctrica	6
2.5. Gemelo digital: concepto y aplicaciones en redes	7
2.6. Trabajos relacionados	8
3. Análisis y Diseño	9
3.1. Requisitos del Sistema	9
3.1.1. Requisitos Funcionales	9
3.1.2. Requisitos No Funcionales	10
3.2. Arquitectura del Sistema	11
3.2.1. Visión General de la Arquitectura	11
3.2.2. Capa de Simulación: Motor MATLAB	12
3.2.3. Capa de Servidor: API REST Spring Boot	13
3.2.4. Capa de Presentación: Aplicación Angular	13

3.3.	Diseño del Modelo de Datos	14
3.3.1.	Esquema JSON de Simulación	14
3.3.2.	Validación de Datos	15
3.4.	Diseño de la API REST	16
3.4.1.	Definición de Endpoints	16
3.4.2.	Códigos de Respuesta HTTP	16
3.4.3.	Documentación OpenAPI	17
3.5.	Diseño de la Interfaz de Usuario	17
3.5.1.	Componentes de la Aplicación	17
3.5.2.	Interacción con el Mapa	18
3.5.3.	Paleta de Colores RSRP	19
3.6.	Decisiones de Diseño	19
3.6.1.	Justificación del Stack Tecnológico	19
3.6.2.	Selección del Modelo de Propagación	20
4.	Implementación	22
4.1.	Implementación de la Simulación en MATLAB	22
4.1.1.	Visión general del motor de simulación	22
4.1.2.	Parámetros del sistema radiante (Bloque 1)	23
4.1.3.	Generación de la cuadrícula de receptores (Bloque 2)	24
4.1.4.	Datos de elevación del terreno (Bloque 3)	24
4.1.5.	Modelo de propagación radioeléctrica (Bloque 4)	25
4.1.6.	Cálculo del RSRP (Bloque 5)	26
4.1.7.	Exportación de resultados a JSON (Bloque 7)	27
4.2.	Backend: API REST con Spring Boot	29
4.2.1.	Estructura del proyecto	29
4.2.2.	Servicio de simulaciones	30
4.2.3.	Controlador REST	31
4.2.4.	Gestión de errores	32
4.2.5.	Configuración y CORS	33
4.2.6.	Tests unitarios	34
4.3.	Frontend: Aplicación Angular	34
4.3.1.	Arquitectura de componentes	34
4.3.2.	Servicio de simulaciones	35
4.3.3.	Componente de mapa	36
4.3.4.	Interfaz de usuario	37

4.3.5. Gestión del estado con <i>signals</i>	38
4.4. Integración del sistema	38
4.4.1. Flujo de datos extremo a extremo	38
4.4.2. Formato de intercambio JSON	39
4.4.3. Configuración del entorno de desarrollo	40
4.4.4. Despliegue	40
5. Validación y Resultados	42
5.1. Diseño del Escenario de Prueba	42
5.2. Resultados de la Simulación MATLAB	42
5.3. Resultados de la Integración Web	42
5.4. Comparación con Software Comercial	42
5.5. Limitaciones Identificadas	42
6. Conclusiones y Trabajo Futuro	43
6.1. Conclusiones Generales	43
6.2. Consecución de Objetivos	43
6.3. Competencias Adquiridas	43
6.4. Aplicabilidad Práctica	43
6.5. Limitaciones del Trabajo	43
6.6. Líneas de Trabajo Futuro	43
6.7. Impacto y Difusión	44
6.8. Reflexión Personal	44
A. Anexo A: Manual de Usuario	45
B. Anexo B: Manual de Instalación	46
C. Anexo C: Código Fuente Destacado	47
D. Anexo D: Especificación API REST	48
E. Anexo E: Resultados Adicionales	49
F. Anexo F: Diagramas UML	50
Bibliografía	51

Índice de figuras

Listado de códigos

Listado de ecuaciones

Índice de tablas

2.1. Comparativa de modelos de propagación radioeléctrica.	5
3.1. Requisitos funcionales del sistema.	10
3.2. Requisitos no funcionales del sistema.	11
3.3. <i>Endpoints</i> de la API REST.	16
3.5. Componentes Angular de la aplicación.	18
3.6. Paleta de colores y umbrales RSRP.	19
3.8. Comparativa de modelos de propagación (escenario URJC Fuenlabrada, 3,5 GHz).	21
4.1. Estructura de bloques del script <code>simulacion_cobertura_5g.m</code>	22
4.3. Parámetros de la estación base (escenario URJC Fuenlabrada).	23
4.4. Clasificación de la cobertura en función del RSRP (3GPP).	27
4.5. Estructura de paquetes del backend.	29
4.6. Endpoints de la API REST.	31
4.7. Resumen de los tests unitarios del backend.	34
4.9. Componentes de la aplicación Angular.	35
4.10. Esquema del fichero JSON de simulación.	39
4.11. Prerrequisitos del entorno de desarrollo.	40

Capítulo 1

Introducción

1.1. Contexto y Motivación

Evolución de redes móviles ($4G \rightarrow 5G \rightarrow 6G$), necesidad de burbujas tácticas en emergencias, gap entre software comercial caro y herramientas accesibles.

1.2. Descripción del Problema

Planificación de cobertura 5G requiere simulación RF costosa, ingenieros necesitan validación preliminar rápida sin licencias de Xirio/Atoll.

1.3. Objetivos del Trabajo

Objetivo general: desarrollar PoC de gemelo digital web para cobertura 5G FR1. 5 objetivos específicos: modelado MATLAB, interoperabilidad JSON, backend Spring Boot, frontend Angular, validación real.

1.4. Metodología y Planificación

Metodología, fases del proyecto, tecnologías seleccionadas (MATLAB, Java, TypeScript).

1.5. Estructura del Documento

Resumen de capítulos 2-6 y anexos.

Capítulo 2

Estado del Arte

2.1. Evolución de las redes 5G NR

La quinta generación de redes móviles (5G NR, *New Radio*) quedó especificada por el *3rd Generation Partnership Project* (3GPP) a partir de la Release 15, publicada en su versión completa en junio de 2018 [3GPP, 2018]. Su diseño rompe con la filosofía de las generaciones anteriores al separar el plano de control del plano de usuario (*Control/User Plane Separation*, CUPS) y al definir desde el inicio una arquitectura de red de acceso radioeléctrico (RAN) preparada para dividirse en unidades funcionales distribuibles: la *Central Unit* (CU), la *Distributed Unit* (DU) y la *Radio Unit* (RU).

Release 15 (diciembre 2017 – junio 2018). Establece la especificación de referencia para 5G NR. Define dos rangos de frecuencia: FR1 (450 MHz–7,125 GHz, denominado coloquialmente «sub-6 GHz») y FR2 (24,25–52,6 GHz, o «mmWave»). La interfaz radio utiliza OFDM con prefijo cíclico tanto en el enlace descendente como en el ascendente, con subportadoras escalables entre 15 kHz y 240 kHz (*numerology* $\mu = 0-4$). La Rel. 15 también introduce los primeros despliegues *Non-Standalone* (NSA), en los que la celda 5G NR comparte el plano de control con una celda LTE anclada, permitiendo lanzar servicios antes de que exista una core 5G propia.

Release 16 (junio 2020). Completa la especificación *Standalone* (SA), donde el núcleo de red es íntegramente 5G Core (5GC). Introduce soporte completo para URLLC (*Ultra-Reliable Low-Latency Communications*), V2X (*Vehicle-to-Everything*) y *positioning* sub-métrico mediante señales de referencia de posicionamiento (PRS). La fiabilidad de enlace ($\text{BLER} \leq 10^{-5}$) y la latencia extremo a extremo (≤ 1 ms en el aire) que habilita esta release son las que permiten pensar en gemelos digitales industriales sincronizados en tiempo real [3GPP, 2020].

Release 17 (marzo 2022). Incorpora 5G para IoT masivo a bajo coste (*Reduced Capability*, RedCap), que reduce la complejidad del terminal a cambio de limitar el ancho de banda a 20 MHz en FR1. También añade soporte para redes no terrestres (NTN, *Non-Terrestrial Networks*): satélites en órbita baja y plataformas estratosféricas (HAPS), que son de especial relevancia para escenarios tácticos en zonas sin infraestructura fija [3GPP, 2022d].

Escenarios de uso y KPIs. El estándar IMT-2020 de la ITU-R agrupa los servicios 5G en tres familias: eMBB (*enhanced Mobile Broadband*, pico > 20 Gbps en DL), URLLC (latencia ≤ 1 ms, disponibilidad 99,999%) y mMTC (*massive Machine-Type Communications*, 10^6 dispositivos/km²). Las métricas de calidad de señal en la interfaz radio más habituales son el RSRP (*Reference Signal Received Power*) definido en 3GPP TS 38.215 [3GPP, 2022b], el RSRQ (*Reference Signal Received Quality*) y el SINR (*Signal-to-Interference-plus-Noise Ratio*). El RSRP es la magnitud primaria que produce la simulación de este TFG.

2.2. Redes tácticas y burbujas 5G desplegadas

Una red táctica 5G es una celda temporal de cobertura reducida que se despliega en minutos sobre un vehículo, un *flycase* portátil o un dron, pensada para escenarios de emergencia, operaciones militares o eventos masivos sin cobertura comercial suficiente. El operador del despliegue controla completamente la core y el acceso radio, lo que elimina la dependencia de la red comercial y permite priorizar el tráfico de misión crítica [Camp and Knightly, 2020].

Las soluciones comerciales más extendidas en el mercado de defensa y protección civil incluyen la *Mobile Edge Computing Box* de Ericsson (basada en gNodeB de interior con core 5G integrada en un rack de 19”), la plataforma de Huawei para redes privadas 5G y el sistema TAC-5G de L3Harris. En España, el programa RAPAZ del Ejército de Tierra explora el uso de burbujas 4G/5G privadas sobre vehículos tácticos ligeros. Todos estos sistemas enfrentan el mismo problema: sin una herramienta de planificación rápida y precisa, el operador táctico no puede estimar la cobertura antes de posicionar la antena.

2.3. Modelos de propagación radioeléctrica

La selección del modelo de propagación es la decisión técnica de mayor impacto en la precisión de cualquier herramienta de planificación de cobertura. A continuación

revisamos los modelos más relevantes para el rango FR1 (sub-6 GHz).

2.3.1. Pérdidas en el espacio libre (FSPL)

La pérdida en espacio libre es la referencia teórica de mínimas pérdidas, asumiendo que no existe ningún obstáculo entre TX y RX y que la propagación sigue únicamente la ley del inverso del cuadrado de la distancia. La fórmula de Friis da:

$$\text{FSPL [dB]} = 20 \log_{10}(d) + 20 \log_{10}(f_c) + 20 \log_{10}\left(\frac{4\pi}{c}\right) \quad (2.1)$$

donde d es la distancia en metros, f_c la frecuencia en Hz y c la velocidad de la luz. A 3,5 GHz, por ejemplo, la FSPL a 1 km es aproximadamente 123 dB. Este valor sirve como cota inferior de las pérdidas reales: cualquier modelo de propagación en exteriores producirá pérdidas mayores porque incorpora el efecto del terreno, los edificios y la difracción.

2.3.2. Modelos Okumura-Hata y COST-231

El modelo de Okumura [Okumura et al., 1968] fue el primer modelo empírico de gran aceptación para redes celulares. Se construyó a partir de extensas campañas de medida en Tokio a 200–1920 MHz. Hata [Hata, 1980] lo transformó en ecuaciones analíticas cerradas, válidas para 150–1500 MHz, que se convirtieron en la norma ITU-R P.529.

El proyecto COST-231 extendió el modelo de Hata hasta los 2000 MHz para dar cobertura a la banda GSM-1800 y UMTS, añadiendo la constante C_m que diferencia el entorno urbano grande ($C_m = 3$ dB) del medio [COST Action 231, 1999]. La expresión resultante (ecuación 4.3 del capítulo de implementación) es rápida y no requiere datos de terreno, lo que la hizo estándar en la planificación de redes 2G/3G. Sin embargo, su rango de validez estricto termina en 2 GHz, y su extrapolación a 3,5 GHz introduce errores de 2–5 dB respecto a medidas reales [3GPP, 2022c].

2.3.3. Modelo Longley-Rice (ITM)

El *Irregular Terrain Model* (ITM), desarrollado por Longley y Rice en 1968 en el ITS de la NTIA (Boulder, Colorado) [Longley and Rice, 1968], es el único de los modelos considerados que calcula las pérdidas a partir del perfil de elevación real del terreno entre TX y RX. Para cada enlace, el modelo obtiene el perfil de alturas con el mismo paso que la cuadrícula de receptores, lo somete a un análisis de difracción múltiple (método de Fresnel extendido) y estima la contribución de la troposfera a

través del parámetro de irregularidad del terreno Δh . El rango de validez es 20 MHz a 20 GHz, lo que lo hace directamente aplicable a la banda n78 (3,5 GHz).

A diferencia de los modelos estocásticos de 3GPP, el ITM es determinista: dado el mismo perfil de terreno, dos simulaciones producen el mismo resultado. Esto es una ventaja para la planificación táctica, donde el operador necesita saber con certeza si un punto concreto del mapa tendrá cobertura, no solo la media estadística de la zona.

El ITM fue revisado y validado en el informe NTIA 82-100 [Hufford et al., 1982] y sigue siendo la referencia del gobierno federal de Estados Unidos para análisis de interferencias (FCC 47 CFR Part 73). En MATLAB está disponible a partir de R2019b mediante `propagationModel('longley-rice')` dentro del *Communications Toolbox*.

2.3.4. 3GPP TR 38.901

El informe técnico 3GPP TR 38.901 [3GPP, 2022c] define los modelos de canal estocástico para 5G NR en el rango 0,5–100 GHz. A diferencia de los modelos anteriores, no produce una pérdida de trayecto única sino una realización estadística de canal completa, con *clusters* de *rays* caracterizados por su retardo, ángulo de salida y llegada y amplitud. Los escenarios definidos son UMi (*Urban Micro*), UMa (*Urban Macro*), RMa (*Rural Macro*) y *Indoor Hotspot*.

Para la herramienta de este TFG no utilizamos TR 38.901 directamente como modelo de propagación porque sus realizaciones estocásticas no son comparables punto a punto con medidas de campo sin promediar sobre muchos snapshots. Sin embargo, sí empleamos sus umbrales de RSRP para clasificar la calidad de cobertura (véase el Cuadro 4.4 en §4.1.6 del capítulo de implementación).

2.3.5. Comparativa de modelos

El Cuadro 2.1 sintetiza las características principales de los cuatro modelos presentados en esta sección, con el fin de justificar la selección del modelo Longley-Rice como motor de propagación del gemelo digital.

Tabla 2.1: Comparativa de modelos de propagación radioeléctrica.

Criterio	FSPL	COST-231	Longley-Rice	TR 38.901
Rango de frecuencia	Cualquiera	1,5–2 GHz	20 MHz–20 GHz	0,5–100 GHz
Rango de distancia	Cualquiera	1–20 km	1–2000 km	<5 km
Datos de terreno	No	No	Sí (SRTM)	Estadístico
Tipo de resultado	Determinista	Determinista	Determinista	Estocástico
Error típico a 3,5 GHz	0 dB (teórico)	± 15 –20 dB	± 6 –10 dB	± 4 –8 dB
Coste computacional	Mínimo	Bajo	Medio	Alto
Uso en este TFG	Referencia	Fallback	Principal	Validación

2.4. Herramientas de planificación radioeléctrica

Existen varias herramientas de planificación de cobertura en el mercado. Las evaluamos desde la perspectiva de su adecuación a un despliegue táctico rápido, donde el operador necesita resultados en menos de cinco minutos con datos de terreno actualizados.

Xirio Online. Plataforma SaaS española de INDRA que ofrece cálculo de cobertura por *ray-tracing* y modelos estadísticos sobre cartografía OSM. Su precisión es elevada, especialmente en entornos urbanos densos, pero el acceso requiere suscripción (~ 5000 €/año) y el cálculo de un mapa de 100 km^2 puede tardar varios minutos en la capa de servidor compartida. No dispone de API pública documentada para integración automatizada [INDRA / Ingeniería de Sistemas para la Defensa de España, 2024].

Atoll (Forsk). Herramienta de escritorio y de red que soporta LTE, 5G NR y Wi-Fi. Es el estándar de facto en los departamentos de ingeniería de los operadores europeos. Gestiona propagación con modelos SPM (*Standard Propagation Model*), Longley-Rice y *ray-tracing*. El coste de licencia (decenas de miles de euros por sede) y la dependencia de Windows la hacen inviable para un TFG.

SPLAT! Herramienta de código abierto para Linux desarrollada por John Magliacane (KD2BD) [Magliacane, 2011], que aplica el modelo Longley-Rice sobre ficheros de terreno SRTM. Permite calcular curvas de cobertura y enlaces punto a punto con resolución de 1 arco-segundo (30 m). Su interfaz de línea de comandos es sencilla pero no ofrece visualización interactiva ni exportación JSON directa, lo que dificulta su integración en un pipeline moderno.

CloudRF. Servicio *freemium* web que expone el motor SPLAT!/ITM a través de una API REST y un *front-end* de mapa. La capa gratuita permite 10 cálculos/día con resolución degradada; la versión de pago soporta alta resolución y *bulk jobs*. La dependencia de conectividad con los servidores del proveedor la descarta para uso táctico en zonas sin acceso a internet [CloudRF, 2024].

RadioMobile. Herramienta gratuita de Roger Coude (VE2DBE) para Windows, orientada a radioaficionados y emergencias. Usa modelos ITM e ITU-R P.370. Su interfaz ha quedado obsoleta y carece de soporte para 5G NR.

Gap identificado. Ninguna de las herramientas anteriores cumple simultáneamente los tres requisitos clave de un entorno táctico desplegable: (1) gratuita y sin dependencia de servidores externos; (2) exportación estandarizada de resultados (JSON) para integración con sistemas de mando y control; y (3) interfaz web moderna usable por un operador sin formación en RF. Este TFG cubre ese hueco.

2.5. Gemelo digital: concepto y aplicaciones en redes

El concepto de *digital twin* (gemelo digital) fue formulado por Michael Grieves en 2002 en el contexto de la gestión del ciclo de vida del producto (PLM) en la industria aeroespacial [Grieves, 2014]. La idea central es la de un par de entidades: el sistema físico real y su contraparte virtual, sincronizadas mediante un flujo de datos bidireccional. La réplica virtual permite simular el comportamiento del sistema en condiciones que sería costoso o peligroso reproducir en el mundo real, anticipar fallos y optimizar parámetros en tiempo prácticamente real.

Fuller et al. [Fuller et al., 2020] ofrecen la revisión sistemática más citada del estado del arte en gemelos digitales, con más de 2000 referencias. Distinguen tres tipos de gemelo según su nivel de fidelidad:

- **Descriptivo:** réplica estática del sistema en un instante dado (modelo CAD, plano de planta).
- **Predictivo:** incluye un modelo dinámico que permite simular el comportamiento futuro (simulación de fluidos, electromagnética).
- **Prescriptivo:** cierra el bucle entre simulación y actuación, de forma que el gemelo propone (o ejecuta) acciones sobre el sistema real para alcanzar un objetivo definido.

En el ámbito de las telecomunicaciones, los gemelos digitales han encontrado aplicación en dos frentes principales. En la **optimización de redes**: operadores como Ericsson y Nokia han presentado plataformas de gemelo digital de red que agregan trazas de OSS/BSS y KPIs de celda para detectar anomalías y sugerir resintonizaciones automáticas de parámetros RAN [ITU-T, 2021]. En la **planificación de despliegue**: grupos de investigación como el de la Universidad de Oulu han propuesto gemelos digitales de zona urbana para seleccionar emplazamientos de small cells con validación antes del despliegue físico.

El gemelo digital de este TFG es de tipo **predictivo**: toma los parámetros de un gNodeB (frecuencia, potencia, emplazamiento, datos SRTM) y genera un mapa de RSRP simulado que predice la cobertura real antes de posicionar la antena física. La interfaz Angular visualiza esa predicción en tiempo real y permite iterar sobre los parámetros de configuración sin necesidad de un despliegue físico, lo que reduce drásticamente el tiempo de planificación táctica.

2.6. Trabajos relacionados

Varios grupos académicos han abordado problemas parcialmente solapados con el de este TFG.

Nguyen et al. [Nguyen et al., 2021] proponen un gemelo digital de red 5G basado en simulación de canal con NS-3, con enfoque en la optimización de *beamforming* masivo (mMIMO). Su plataforma es muy precisa en entornos estáticos conocidos pero no aborda el problema de la planificación rápida sobre terreno desconocido.

Alraih et al. [Alraih et al., 2022] realizan una revisión de modelos de propagación para bandas superiores a 6 GHz (*millimeter-wave*) relevantes para 5G y mmWave. Si bien este TFG se centra en FR1 (sub-6 GHz), algunos de sus resultados sobre la sensibilidad del error de predicción al modelo elegido son directamente aplicables.

En el ámbito de herramientas abiertas, Gast et al. describen el uso de SPLAT! para planificación de redes de emergencia 802.11 sobre SRTM y validan el modelo frente a medidas GPS en campo [Gast, 2005]. Su metodología de validación (RMSE de RSSI frente a distancia) es la referencia directa para el protocolo de validación experimental del modelo propuesto en este TFG (Cap. 5).

Diferenciación de este TFG. Frente a los trabajos anteriores, la contribución de este proyecto combina cuatro elementos que no aparecen juntos en la literatura revisada: (1) integración del modelo Longley-Rice con datos SRTM de 30 m en un script MATLAB completamente reproducible; (2) API REST Spring Boot como capa de servicio; (3) visualización Angular con heatmap interactivo y selección de punto en el mapa; y (4) foco explícito en el escenario de burbuja táctica desplegable, con especial atención a la usabilidad del operador en campo. El código es abierto y reproducible bajo licencia MIT.

Capítulo 3

Análisis y Diseño

Este capítulo describe las decisiones de análisis y diseño que guían la construcción del gemelo digital de cobertura 5G. Se parte de la elicitación de requisitos funcionales y no funcionales, se define la arquitectura en tres capas del sistema, se detalla el modelo de datos JSON que conecta la simulación con la visualización y se expone el diseño de la API REST y de la interfaz de usuario. El capítulo cierra con la justificación razonada del *stack* tecnológico y del modelo de propagación seleccionado.

3.1. Requisitos del Sistema

La especificación de requisitos se ha elaborado siguiendo un enfoque incremental: los requisitos iniciales se extrajeron de las necesidades del caso de uso táctico descrito en el Capítulo 2 y se refinaron durante las iteraciones de desarrollo. Se separan en requisitos funcionales (qué debe hacer el sistema) y requisitos no funcionales (con qué restricciones de calidad debe hacerlo).

3.1.1. Requisitos Funcionales

Los requisitos funcionales definen las capacidades que el sistema debe proporcionar al usuario final —el operador táctico que planifica el despliegue de una burbuja 5G— y al ingeniero de telecomunicaciones que analiza los resultados de la simulación.

Tabla 3.1: Requisitos funcionales del sistema.

ID	Descripción	Prioridad
RF1	El sistema debe simular la cobertura 5G FR1 (banda n78, 3,5 GHz) aplicando el modelo de propagación Longley-Rice ITM sobre datos de elevación SRTM de 30 m de resolución.	Alta
RF2	Los resultados de cada simulación deben exportarse en formato JSON con un identificador UUID v4 único, incluyendo parámetros de configuración, metadatos y los puntos de cobertura con coordenadas y valores RSRP.	Alta
RF3	El servidor debe exponer una API REST que permita consultar el listado de simulaciones disponibles y obtener el detalle completo de cualquiera de ellas mediante su UUID.	Alta
RF4	La aplicación web debe visualizar los puntos de cobertura de una simulación como un <i>heatmap</i> interactivo superpuesto sobre un mapa cartográfico, con colores representativos de los niveles de RSRP.	Alta
RF5	El usuario debe poder filtrar de forma interactiva los puntos de cobertura por umbral de RSRP mediante un control deslizante (<i>slider</i>), actualizándose el mapa en tiempo real.	Media

3.1.2. Requisitos No Funcionales

Los requisitos no funcionales establecen las restricciones de calidad, rendimiento y compatibilidad que el sistema debe satisfacer para ser aceptable en un entorno de uso táctico real, donde la rapidez y la fiabilidad son críticas.

Tabla 3.2: Requisitos no funcionales del sistema.

ID	Descripción	Categoría
RNF1	La interfaz de usuario debe ser responsiva, adaptándose correctamente a resoluciones de escritorio (≥ 1280 px) y tableta (≥ 768 px).	Usabilidad
RNF2	El tiempo de carga y renderizado del <i>heatmap</i> para una simulación de 3600 puntos no debe superar los 3 segundos en un equipo de gama media con conexión local.	Rendimiento
RNF3	El backend debe ser capaz de servir entre 5 y 10 ficheros de simulación simultáneamente sin degradación perceptible del rendimiento.	Escalabilidad
RNF4	La aplicación web debe funcionar correctamente en las dos últimas versiones estables de Google Chrome, Mozilla Firefox y Microsoft Edge.	Compatibilidad
RNF5	La cobertura de tests unitarios y de integración del backend debe ser superior al 60 % del código fuente.	Mantenibilidad

3.2. Arquitectura del Sistema

La arquitectura del sistema sigue un patrón de tres capas desacopladas, cada una con una responsabilidad bien definida: simulación, servicio de datos y presentación. Este diseño permite que cada capa evolucione de forma independiente y facilita la sustitución de componentes sin impacto en el resto del pipeline.

3.2.1. Visión General de la Arquitectura

El flujo de datos del gemelo digital es unidireccional y consta de cinco etapas, como se muestra a continuación:

1. **Simulación** (MATLAB, ejecución *offline*): el motor de simulación calcula la cobertura RSRP sobre una cuadrícula de 60×60 puntos utilizando el modelo Longley-Rice ITM con datos de elevación SRTM. El resultado se serializa en un fichero JSON nombrado con un UUID v4.
2. **Almacenamiento**: los ficheros JSON se depositan en un directorio compartido (`matlab/output/`) que actúa como almacén persistente del sistema. No se emplea base de datos relacional: el fichero JSON *es* la fuente de verdad.
3. **Servicio** (Spring Boot): la API REST lee los ficheros JSON del directorio configurado, los deserializa con Jackson y los expone mediante *endpoints* RESTful documentados con OpenAPI.

4. **Transporte:** las peticiones HTTP entre frontend y backend se realizan sobre `localhost` en desarrollo (puertos 4200 y 8080 respectivamente), con CORS habilitado explícitamente.
5. **Presentación** (Angular + Leaflet): la aplicación web consume la API, transforma los datos al modelo interno y renderiza el *heatmap* interactivo sobre un mapa OpenStreetMap.

Este diseño desacoplado tiene una ventaja importante para el caso de uso táctico: la capa de simulación puede ejecutarse en un portátil con MATLAB sin conexión a internet, generar los JSON con los resultados y, posteriormente, cargarlos en un servidor Spring Boot ligero que sirva la visualización a cualquier dispositivo conectado a la red local de la burbuja 5G. No se requiere conectividad externa en ningún punto del pipeline.

3.2.2. Capa de Simulación: Motor MATLAB

El motor de simulación es un script MATLAB (`simulacion_cobertura_5g.m`) estructurado en nueve bloques ejecutables de forma independiente. Su responsabilidad es triple: (1) configurar los parámetros de la estación base y la cuadrícula de receptores; (2) seleccionar automáticamente el mejor modelo de propagación disponible en función de los *toolboxes* instalados; y (3) exportar los resultados en formato JSON compatible con el backend.

La selección de modelo sigue una lógica de degradación controlada. Si el *Communications Toolbox* está disponible, se utiliza el modelo Longley-Rice ITM, que incorpora el perfil de elevación real del terreno obtenido de datos SRTM a 30 m de resolución. Si el *toolbox* no está disponible, el script recurre al modelo COST-231 Hata como *fallback*, registrando una advertencia en los metadatos del JSON de salida. En ambos casos se calcula también la pérdida en espacio libre (FSPL) como cota superior teórica de validación.

La integración de datos de elevación se realiza mediante el *Mapping Toolbox*: las funciones `readgeoraster` y `geointerp` cargan el *tile* SRTM (GeoTIFF) e interpolan la altitud en cada punto de la cuadrícula. Si el fichero SRTM no está disponible, el sistema asume terreno plano y continúa con la simulación, marcando el campo `terrain_data` del JSON como `"flat_assumed"`.

3.2.3. Capa de Servidor: API REST Spring Boot

El backend está implementado con Spring Boot 3.2.4 sobre Java 17, utilizando Maven como gestor de dependencias. Su función es exponer los resultados de las simulaciones MATLAB a través de una API REST *stateless*, sin persistencia en base de datos: los ficheros JSON del directorio de salida de MATLAB son la única fuente de datos.

La estructura de paquetes sigue las convenciones estándar de Spring Boot:

- `controller/`: clase `SimulationController` con los *endpoints* REST anotados con `@RestController`.
- `service/`: clase `SimulationService` que encapsula la lógica de lectura y deserialización de ficheros JSON.
- `model/`: *records* Java (`SimulationSummaryDto`, etc.) que representan los DTOs de respuesta.
- `exception/`: clase `SimulationNotFoundException` anotada con `@ResponseStatus(HttpStatus.NOT_FOUND)` y manejador global de excepciones.
- `config/`: clase `CorsConfig` que habilita las peticiones *cross-origin* desde `localhost:4200`.

El diseño RESTful es completamente *stateless*: cada petición contiene toda la información necesaria para ser procesada, no se mantiene estado de sesión en el servidor y todos los *endpoints* son idempotentes (operaciones GET exclusivamente). La documentación interactiva de la API se genera automáticamente mediante `springdoc-openapi 2.3.0`, accesible en `/swagger-ui.html`.

3.2.4. Capa de Presentación: Aplicación Angular

La interfaz de usuario está desarrollada con Angular 17+ utilizando componentes *standalone* (sin `NgModule`), lo que simplifica la estructura del proyecto y mejora el *tree-shaking* del compilador. La aplicación consume la API REST del backend y presenta los datos de cobertura como un *heatmap* interactivo renderizado sobre un mapa OpenStreetMap mediante las bibliotecas Leaflet y `leaflet.heat`.

La gestión de estado se basa en *signals* de Angular, un mecanismo reactivo introducido en Angular 16 que permite propagar cambios de forma eficiente sin necesidad de bibliotecas externas como `NgRx` o `RxJS` para el estado local. El servicio

`SimulationService` expone *signals* de solo lectura para el estado de carga (`loading`), los errores (`error`) y la lista de simulaciones disponibles.

La comunicación con el backend se realiza mediante `HttpClient` de Angular, inyectado mediante `provideHttpClient()` en la configuración de arranque de la aplicación. Los DTOs del backend (`ApiSimulation`, `ApiCoveragePoint`) se transforman al modelo interno de la aplicación mediante la función `apiToSimulation()`, desacoplando así la representación del servidor de la representación de la vista.

3.3. Diseño del Modelo de Datos

El fichero JSON constituye el contrato de datos entre la capa de simulación y el resto del sistema. Su estructura ha sido diseñada para ser autocontenida: cada fichero incluye tanto los parámetros de configuración de la simulación como los resultados, de forma que puede interpretarse sin información adicional externa.

3.3.1. Esquema JSON de Simulación

Cada simulación se almacena en un fichero JSON independiente, nombrado con su UUID v4 (por ejemplo, `b83baf94-8db6-4512-a34b-b831e3f6267a.json`). El esquema del fichero se muestra en el Listado 3.1.

Listing 3.1: Estructura del fichero JSON de simulación.

```
1 {
2   "id": "b83baf94-8db6-4512-a34b-b831e3f6267a",
3   "name": "URJC Fuenlabrada - Longley-Rice ITM",
4   "createdAt": "2026-04-10T14:32:00Z",
5   "parameters": {
6     "frequency_hz": 3500000000,
7     "tx_power_dbm": 40,
8     "antenna_height_m": 30,
9     "rx_height_m": 1.5,
10    "tx_latitude": 40.2897,
11    "tx_longitude": -3.8244,
12    "area_km": 3,
13    "espaciado_grid_m": 50
14  },
15  "metadata": {
16    "frequency_ghz": 3.5,
17    "tx_power_dbm": 40,
18    "antenna_height_m": 30,
19    "tx_location": { "latitude": 40.2897, "longitude": -3.8244 },
20    "simulation_date": "2026-04-10",
21    "model": "longley-rice",
22    "terrain_data": "srtm_30m"
23  },
24  "coveragePoints": [
25    { "lat": 40.2762, "lng": -3.8379, "rsrp": -97.3 },
26    { "lat": 40.2762, "lng": -3.8374, "rsrp": -95.1 },
```

```

27     ...
28   ]
29 }

```

Los campos principales son:

- **id**: identificador UUID v4 generado por MATLAB en el momento de la exportación. Sirve como clave primaria en el backend y como nombre del fichero.
- **parameters**: parámetros de configuración de la simulación (frecuencia, potencia, alturas de antena, coordenadas del emplazamiento y geometría de la cuadrícula).
- **metadata**: información descriptiva para el frontend (modelo utilizado, tipo de datos de terreno, fecha de ejecución).
- **coveragePoints**: array de objetos con las coordenadas geográficas (**lat**, **lng**) y el valor RSRP en dBm de cada punto de la cuadrícula. Una simulación estándar contiene 3600 puntos.

3.3.2. Validación de Datos

La integridad de los datos se garantiza mediante validaciones en dos niveles del sistema.

En la **capa de simulación**, MATLAB aplica restricciones sobre los valores generados antes de la exportación. Los valores de RSRP se recortan al rango $[-140, -40]$ dBm, que corresponde a los límites definidos en 3GPP TS 38.215 [3GPP, 2022b] para medidas de potencia de señal de referencia. La frecuencia de simulación se restringe al rango de la banda n78 (3,4–3,8 GHz), verificándose que el valor configurado en **fc_Hz** caiga dentro de este intervalo.

En la **capa de servidor**, el backend Spring Boot aplica validaciones adicionales sobre las peticiones entrantes:

- **Validación de UUID**: el parámetro de ruta `{uuid}` se valida contra una expresión regular que verifica el formato UUID v4 estándar (8-4-4-4-12 dígitos hexadecimales con el nibble de versión fijado a 4). Las peticiones con formato incorrecto reciben una respuesta **404 Not Found**.
- **Protección contra *path traversal***: el servicio verifica que el nombre del fichero solicitado no contenga secuencias como `../` que pudieran permitir el acceso a ficheros fuera del directorio de simulaciones configurado. Los intentos de *path traversal* se rechazan con **404 Not Found** sin revelar información sobre la estructura del sistema de ficheros.

3.4. Diseño de la API REST

La API REST constituye la interfaz de comunicación entre el backend y cualquier cliente que desee consumir los resultados de las simulaciones. El diseño sigue los principios REST clásicos: recursos identificados por URI, operaciones sin estado y representaciones en JSON.

3.4.1. Definición de Endpoints

El Cuadro 4.6 resume los *endpoints* disponibles en la API. Todos ellos son operaciones de solo lectura (**GET**), coherente con el hecho de que las simulaciones se generan en MATLAB y el backend actúa exclusivamente como capa de consulta.

Tabla 3.3: *Endpoints* de la API REST.

Endpoint	Descripción	Resp.
GET /api/simulations	Devuelve el listado de simulaciones disponibles, incluyendo <code>id</code> , <code>name</code> , <code>createdAt</code> y <code>metadata</code> . No incluye los <code>coveragePoints</code> para minimizar el tamaño de la respuesta. Soporta paginación opcional con los parámetros <code>?page=0&size=10</code> .	200
GET /api/simulations/{uuid}	Devuelve el detalle completo de una simulación, incluyendo los <code>coveragePoints</code> (hasta 3600 objetos).	200 / 404
GET /api/simulations/{uuid}/export	Descarga el fichero JSON original de la simulación como adjunto (<code>Content-Disposition: attachment</code>).	200 / 404
GET /swagger-ui.html	Documentación interactiva OpenAPI generada automáticamente por <code>springdoc-openapi</code> .	200

3.4.2. Códigos de Respuesta HTTP

La API utiliza un conjunto reducido de códigos de respuesta HTTP, suficiente para cubrir los casos de uso de un servicio de solo lectura:

- **200 OK:** la petición se ha procesado correctamente. El cuerpo contiene el recurso solicitado en formato JSON.

- **404 Not Found:** el UUID proporcionado no corresponde a ninguna simulación existente, tiene un formato inválido o el intento de acceso ha sido rechazado por la protección de *path traversal*. El cuerpo contiene un mensaje de error genérico que no revela detalles internos.
- **500 Internal Server Error:** error inesperado del servidor (por ejemplo, fallo de lectura del sistema de ficheros). El manejador global de excepciones (`@ControllerAdvice`) captura la excepción y devuelve un cuerpo JSON estructurado con el *timestamp* y el mensaje de error.

3.4.3. Documentación OpenAPI

La documentación de la API se genera automáticamente mediante la biblioteca `springdoc-openapi` en su versión 2.3.0, declarada como dependencia Maven en el fichero `pom.xml` del proyecto. Esta biblioteca escanea las anotaciones de Spring (`@RestController`, `@GetMapping`, `@PathVariable`) y genera una especificación OpenAPI 3.0 en tiempo de ejecución, accesible tanto en formato JSON (`/v3/api-docs`) como a través de la interfaz gráfica Swagger UI en `/swagger-ui.html`.

La documentación interactiva permite al desarrollador o al integrador probar los *endpoints* directamente desde el navegador, visualizar los esquemas de respuesta y consultar los códigos de error posibles. Esto elimina la necesidad de mantener una documentación separada que pueda quedar desactualizada respecto al código fuente.

3.5. Diseño de la Interfaz de Usuario

La interfaz de usuario sigue una disposición de panel lateral con mapa principal, optimizada para que el operador pueda seleccionar y comparar simulaciones sin perder de vista el mapa de cobertura. El diseño se ha validado con *mockups* de alta fidelidad antes de la implementación.

3.5.1. Componentes de la Aplicación

La aplicación Angular se estructura en siete componentes *standalone*, cada uno con una responsabilidad claramente definida. El Cuadro 3.5 los describe.

Tabla 3.5: Componentes Angular de la aplicación.

Componente	Responsabilidad
MapComponent	Inicializa el mapa Leaflet, gestiona las capas de <i>tiles</i> OpenStreetMap y renderiza la capa de calor (<i>heatmap</i>) mediante <code>leaflet.heat</code> .
SidebarComponent	Panel lateral con el selector de simulaciones, los controles de filtrado y el acceso a la configuración de visualización.
HeaderComponent	Barra superior con el nombre de la aplicación, el logotipo y el botón de alternancia de tema claro/oscuro.
InfoPanelComponent	Panel flotante que muestra los detalles del punto seleccionado al hacer clic sobre el mapa: coordenadas, valor RSRP y clasificación de cobertura.
StatsPanelComponent	Muestra estadísticas agregadas de la simulación activa: RSRP medio, mínimo y máximo, número de puntos por zona de cobertura y porcentaje de área cubierta.
CoverageLegendComponent	Leyenda visual con la escala de colores RSRP, indicando los umbrales y la clasificación de calidad de cada zona.
SimulationConfigComponent	Muestra los parámetros de la simulación seleccionada: frecuencia, potencia, modelo de propagación, fecha de ejecución y tipo de datos de terreno.

Además de los componentes visuales, la aplicación cuenta con dos servicios inyectables: `SimulationService`, que encapsula las llamadas HTTP al backend y mantiene el estado reactivo mediante *signals*, y `ThemeService`, que gestiona la alternancia entre tema claro y oscuro persistiendo la preferencia del usuario.

3.5.2. Interacción con el Mapa

El mapa constituye el elemento central de la interfaz y ofrece tres formas de interacción:

1. **Inspección por clic:** al hacer clic sobre cualquier punto del *heatmap*, se activa el `InfoPanelComponent`, que muestra las coordenadas geográficas del punto más cercano y su valor RSRP en dBm, junto con la clasificación de cobertura correspondiente.
2. **Actualización automática:** cuando el usuario selecciona una simulación diferente en el `SidebarComponent`, el *heatmap* se recalcula y rerenderiza

automáticamente. El componente de mapa destruye la capa de calor anterior, crea una nueva con los datos de la simulación seleccionada y ajusta la vista del mapa al *bounding box* de los nuevos puntos.

3. **Filtrado por umbral RSRP:** un control deslizante (*slider*) permite al usuario establecer un umbral mínimo de RSRP. Los puntos con valores inferiores al umbral se ocultan del *heatmap*, permitiendo visualizar únicamente las zonas con calidad de señal suficiente para un servicio concreto.

3.5.3. Paleta de Colores RSRP

La escala de colores utilizada para representar los niveles de RSRP en el *heatmap* se ha diseñado siguiendo las convenciones habituales en herramientas de planificación radioeléctrica, con una progresión intuitiva de verde (señal fuerte) a rojo oscuro (sin cobertura). Los umbrales se basan en las recomendaciones de 3GPP TS 38.215 [3GPP, 2022b] y en la práctica habitual de los operadores europeos.

Tabla 3.6: Paleta de colores y umbrales RSRP.

Umbral RSRP	Color	Clasificación	Descripción
≥ -80 dBm	Verde	Excelente	Señal óptima; todos los servicios 5G disponibles (eMBB, URLLC).
-90 a -80 dBm	Amarillo-verde	Buena	Calidad de servicio estable para la mayoría de aplicaciones.
-100 a -90 dBm	Naranja	Aceptable	Servicio básico de datos disponible; posible degradación en servicios de baja latencia.
-110 a -100 dBm	Rojo	Débil	Señal muy débil; conexión inestable y alto riesgo de pérdida de enlace.
< -110 dBm	Rojo oscuro	Sin cobertura	Sin servicio 5G utilizable; por debajo del umbral de sensibilidad del receptor.

3.6. Decisiones de Diseño

Esta sección documenta las decisiones técnicas más relevantes tomadas durante la fase de diseño, junto con su justificación.

3.6.1. Justificación del Stack Tecnológico

La elección de cada tecnología responde a criterios de adecuación funcional, madurez del ecosistema y disponibilidad de herramientas de desarrollo:

- **MATLAB** para la capa de simulación: MATLAB dispone del *Communications Toolbox*, que implementa el modelo Longley-Rice ITM de forma nativa mediante `propagationModel('longley-ric')`, junto con funciones de creación de emplazamientos (`txsite`, `rxsite`) y cálculo de potencia recibida (`sigstrength`). El *Mapping Toolbox* proporciona funciones de carga e interpolación de datos SRTM. Ningún otro entorno ofrece una integración equivalente entre modelado de propagación y datos de elevación con la misma productividad.
- **Spring Boot 3.2 + Java 17** para el backend: Spring Boot es el *framework* Java más utilizado para servicios REST, con un ecosistema maduro de *starters*, gestión automática de dependencias mediante Maven y generación de documentación OpenAPI con `springdoc-openapi`. Java 17, al ser una versión LTS (*Long-Term Support*), garantiza estabilidad y soporte a largo plazo. Los *records* de Java 16+ permiten definir DTOs inmutables de forma concisa.
- **Angular 17+** para el frontend: Angular ofrece tipado estático mediante TypeScript, lo que reduce los errores en tiempo de ejecución y mejora la refactorización del código. Los componentes *standalone* introducidos en Angular 14 y consolidados en Angular 17 simplifican la arquitectura del proyecto. El sistema de *signals* proporciona reactividad eficiente sin dependencias externas de gestión de estado.
- **Leaflet + leaflet.heat** para la visualización cartográfica: Leaflet es la biblioteca de mapas web de código abierto más ligera y extendida, con un tamaño de apenas 42 kB. El *plugin* `leaflet.heat` añade capacidad de renderizado de *heatmaps* con un API mínimo, ideal para representar los 3600 puntos de cobertura RSRP sin sobrecarga de rendimiento.

3.6.2. Selección del Modelo de Propagación

La selección del modelo de propagación es la decisión de diseño de mayor impacto técnico en la precisión del gemelo digital. Se evaluaron tres modelos en un escenario de referencia (campus URJC de Fuenlabrada, cuadrícula 60×60 , frecuencia 3,5 GHz, EIRP 53 dBm). Los resultados completos de la comparativa se presentan en el Capítulo 5; aquí se resumen las conclusiones que fundamentan la decisión de diseño.

FSPL (*Free Space Path Loss*). Se utiliza únicamente como referencia teórica de cota superior. Al no incorporar terreno ni obstáculos, produce valores de potencia

recibida irrealmente elevados (hasta $-21,4$ dBm en zona cercana). No es un modelo válido para planificación real, pero resulta útil como prueba de coherencia: cualquier modelo realista debe producir pérdidas mayores que la FSPL.

COST-231 Hata. Este modelo empírico está validado para el rango 1500–2000 MHz. Su extrapolación a 3,5 GHz introduce un error sistemático de +15 a +20 dB en pérdidas, lo que produce resultados catastróficos: en el escenario de referencia, el 95,2 % del área aparece sin cobertura, un resultado incompatible con la realidad de un emplazamiento con EIRP de 53 dBm a distancias inferiores a 1,5 km. Se mantiene en el sistema exclusivamente como *fallback* de emergencia para entornos MATLAB sin *Communications Toolbox*, con advertencia explícita en los metadatos del JSON generado.

Longley-Rice ITM (modelo seleccionado). El *Irregular Terrain Model* [Longley and Rice, 1968], validado por la NTIA para el rango 20 MHz–20 GHz [Hufford et al., 1982], es el único de los tres modelos que incorpora el perfil real de elevación del terreno en el cálculo de pérdidas. En el escenario de referencia produce un RSRP medio de $-93,9$ dBm, con un 93,4 % de los puntos por encima del umbral de cobertura aceptable (-100 dBm), lo que es coherente con las expectativas para una macrocelda 5G suburbana. El Cuadro 3.8 resume la comparativa.

Tabla 3.8: Comparativa de modelos de propagación (escenario URJC Fuenlabrada, 3,5 GHz).

Criterio	FSPL	COST-231	Longley-Rice
Validez a 3,5 GHz	Sí (teórico)	No (± 15 dB)	Sí
Incorpora terreno	No	No	Sí
Coherencia con TR 38.901	Referencia	Parcial	Alta
Disponibilidad MATLAB	Siempre	Siempre	Comm. Toolbox
RSRP medio	—	$-125,8$ dBm	$-93,9$ dBm
Cobertura ≥ -100 dBm	—	1,4 %	93,4 %
Rol en el sistema	Validación	<i>Fallback</i>	Principal

La decisión de diseño es, por tanto, utilizar el modelo Longley-Rice ITM como modelo principal del gemelo digital, con degradación automática a COST-231 Hata —debidamente señalizada— si el *Communications Toolbox* no está disponible en el entorno de ejecución.

Capítulo 4

Implementación

4.1. Implementación de la Simulación en MATLAB

4.1.1. Visión general del motor de simulación

La simulación radioeléctrica vive en dos ficheros: el script principal `simulacion_cobertura_5g.m` y la función `exportar_json.m`, que se ocupa de serializar la salida. Entre los dos calculan el mapa de cobertura de un *gNodeB* 5G NR sobre una cuadrícula de 3×3 km y lo exportan en JSON para que el backend Spring Boot lo sirva sin ningún paso intermedio manual.

Decidimos dividir el script en nueve bloques ejecutables de forma independiente porque durante el desarrollo fue clave poder validar cada paso por separado antes de encadenarlos. El Cuadro 4.1 los resume.

Tabla 4.1: Estructura de bloques del script `simulacion_cobertura_5g.m`.

Bloque	Nombre	Función
0	Limpieza del entorno	<code>clear, clc, close all</code>
1	Parámetros de configuración	Define antena, frecuencia, potencia y cuadrícula
2	Cuadrícula de receptores	Genera el <i>meshgrid</i> de puntos de muestreo
3	Elevación SRTM	Carga e interpola el MDT en la cuadrícula
4	Modelo de propagación	Calcula P_{rx} en cada punto (LR-ITM o COST-231)
5	Cálculo del RSRP	Convierte P_{rx} a RSRP según 3GPP TS 38.215
6	Visualización	Genera <i>heatmaps</i> y estadísticas de cobertura
7	Exportación a JSON	Serializa los resultados para el backend
8	Validación con FSPL	Comprueba coherencia con la cota teórica

El flujo es lineal: parámetros $\rightarrow$ cuadrícula $\rightarrow$ elevación $\rightarrow P_{rx} \rightarrow$ RSRP $\rightarrow$ JSON. El fichero de salida, nombrado con un UUID v4, es el único artefacto que cruza la frontera hacia el resto del sistema.

4.1.2. Parámetros del sistema radiante (Bloque 1)

Todos los parámetros configurables están agrupados al inicio del script para que sea fácil lanzar distintos escenarios sin buscar constantes dispersas por el código. El escenario de referencia modela un *gNodeB* típico de la banda FR1 n78 (3,4–3,8 GHz), que es la banda de despliegue masivo de 5G en Europa, emplazado en el edificio de Rectorado del campus URJC de Fuenlabrada. Los valores del Cuadro 4.3 son representativos de una instalación real en azotea, no teóricos.

Tabla 4.3: Parámetros de la estación base (escenario URJC Fuenlabrada).

Parámetro	Símbolo	Valor
Frecuencia central	f_c	3,5 GHz (banda n78)
Potencia de transmisión	P_{tx}	40 dBm (10 W)
Ganancia antena transmisora	G_{tx}	15 dBi (sectorial)
Ganancia antena receptora	G_{rx}	0 dBi (omnidireccional)
Pérdidas de cable	L_{cable}	2 dB
Altura antena transmisora	h_{Tx}	30 m
Altura antena receptora	h_{Rx}	1,5 m
Emplazamiento (latitud)	ϕ_0	40,2897° N
Emplazamiento (longitud)	λ_0	3,8244° W

Con estos valores, la EIRP queda en:

$$\text{EIRP} = P_{tx} + G_{tx} - L_{cable} = 40 + 15 - 2 = 53 \text{ dBm} \quad (4.1)$$

Para la cuadrícula optamos por 50 m de espaciado como compromiso entre resolución y tiempo de cómputo: con 3 km de lado obtenemos $60 \times 60 = 3600$ puntos, suficientes para ver gradientes a escala de manzana, y el cálculo Longley-Rice completo tarda unos 60 s en un portátil de gama media.

Listing 4.1: Parámetros de configuración de la simulación (Bloque 1).

```

1 fc_Hz      = 3.5e9;          % Frecuencia central [Hz] - banda n78
2 Ptx_dBm    = 40;            % Potencia de transmisin [dBm] = 10 W
3 Ptx_W      = 10^((Ptx_dBm - 30) / 10);
4 Gtx_dBi    = 15;            % Ganancia antena Tx [dBi]
5 Grx_dBi    = 0;             % Ganancia antena Rx [dBi]
6 hTx_m      = 30;            % Altura antena Tx [m]
7 hRx_m      = 1.5;           % Altura antena Rx [m]
8 L_cable_dB = 2;             % Prdidas de cable [dB]
9 EIRP_dBm   = Ptx_dBm + Gtx_dBi - L_cable_dB;
10
11 lat0 = 40.2897; lon0 = -3.8244; % Campus URJC Fuenlabrada
12 areaKm    = 3;             % Lado del rea cuadrada [km]
13 gridSpacing_m = 50;        % Espaciado de la cuadrícula [m]
14 nPuntos    = areaKm * 1000 / gridSpacing_m; % 60

```

4.1.3. Generación de la cuadrícula de receptores (Bloque 2)

Para situar los 3600 receptores sobre el mapa hay que convertir el área en kilómetros a incrementos en grados geográficos. Usamos la aproximación lineal estándar para latitudes medias:

$$\Delta\phi = \frac{a/2}{111} \text{ grados}, \quad \Delta\lambda = \frac{a/2}{111 \cdot \cos \phi_0} \text{ grados} \quad (4.2)$$

donde $a = 3$ km. El error que introduce esta aproximación en la zona de Fuenlabrada ($\phi_0 \approx 40$) es menor del 0,1 %, perfectamente asumible para nuestro propósito.

Con `linspace` construimos los vectores de coordenadas y con `meshgrid` la cuadrícula 2D. Después aplanamos las matrices `LATS` y `LONS` a vectores columna de 3600 elementos, que entran directamente en los bloques de elevación y propagación.

Listing 4.2: Generación de la cuadrícula de receptores (Bloque 2).

```

1 deltaLat_deg = (areaKm / 2) / 111;
2 deltaLon_deg = (areaKm / 2) / (111 * cosd(lat0));
3
4 lats_vec = linspace(lat0 - deltaLat_deg, lat0 + deltaLat_deg, nPuntos);
5 lons_vec = linspace(lon0 - deltaLon_deg, lon0 + deltaLon_deg, nPuntos);
6
7 [LONS, LATS] = meshgrid(lons_vec, lats_vec);
8 lats_flat = LATS(:); % Vectorizar: 3600 x 1
9 lons_flat = LONS(:);

```

4.1.4. Datos de elevación del terreno (Bloque 3)

Los datos de elevación vienen del SRTM (NASA/USGS). Elegimos el *tile* N40W004 porque cubre el área de Fuenlabrada con 30 m de resolución y se descarga gratis desde USGS EarthExplorer en formato GeoTIFF. Almacenamos el fichero en `matlab/data/srtm_fuenlabrada.tif` para tenerlo junto al script.

Para interpolar la elevación en cada punto de la cuadrícula usamos `readgeoraster` y `geointerp` del *Mapping Toolbox*:

Listing 4.3: Carga e interpolación del MDT SRTM (Bloque 3).

```

1 [elev_data, R] = readgeoraster(srtm_file);
2 elev_data = double(elev_data);
3 elev_data(elev_data < -100) = 0; % Eliminar valores sin dato
4 elev_grid = geointerp(elev_data, R, lats_flat, lons_flat, 'linear');

```

Si el fichero no está disponible, el script avisa con un *warning* y sustituye `elev_grid` por ceros, asumiendo terreno plano. Esto resultó muy útil en las primeras fases del desarrollo, cuando todavía no habíamos descargado el tile. La variable `tiene_srtm`

registra cuál de los dos caminos se tomó y queda reflejada en el campo "terrain_data" del JSON de salida.

4.1.5. Modelo de propagación radioeléctrica (Bloque 4)

Este es el bloque central: calcula P_{rx} en los 3600 puntos aplicando un modelo de propagación semiempírico. Al arrancar, el script comprueba qué *toolboxes* están instalados y selecciona el mejor disponible automáticamente.

Modelo Longley-Rice (ITM)

Elegimos el *Irregular Terrain Model* (ITM) o modelo Longley-Rice [Longley and Rice, 1968] como modelo principal porque es el único de los disponibles en MATLAB que incorpora el perfil real del terreno en el cálculo de pérdidas. A diferencia de los modelos puramente empíricos, evalúa la difracción sobre las irregularidades del terreno (parametrizada por Δh), la reflexión en el suelo y la curvatura de la Tierra para cada enlace TX-RX individual. Para redes desplegables en entornos con orografía variable, ignorar eso sería un error de bulto.

El modelo, desarrollado por A. G. Longley e Y. T. Rice en 1968 en el ITS de la NTIA, lleva décadas siendo la referencia en planificación de redes de radiodifusión y telecomunicaciones militares. En MATLAB está disponible desde R2019b a través del *Communications Toolbox*:

Listing 4.4: Cálculo con Longley-Rice ITM (Bloque 4).

```

1 tx = txsite('Name', 'gNodeB_URJC', ...
2   'Latitude', lat0, 'Longitude', lon0, ...
3   'AntennaHeight', hTx_m, 'TransmitterFrequency', fc_Hz, ...
4   'TransmitterPower', Ptx_W);
5
6 pm = propagationModel('longley-rice');
7
8 rx_sites = rxsite('Latitude', lats_flat, ...
9   'Longitude', lons_flat, ...
10  'AntennaHeight', hRx_m);
11
12 Prx_dBm = sigstrength(rx_sites, tx, pm);

```

`sigstrength` devuelve P_{rx} en dBm para cada receptor usando internamente un MDT SRTM a 90 m de resolución que el *toolbox* lleva de serie. Cuando cargamos nuestro tile local de 30 m a través de un `siteviewer`, ese MDT global queda sustituido y la precisión en la zona de estudio mejora notablemente. Integrar el tile propio es, de hecho, la mejora técnica central de este TFG frente a usar el modelo con su terreno

por defecto.

COST-231 Hata como modelo de respaldo

Cuando el *Communications Toolbox* no está instalado, el script cae en COST-231 Hata extendido [COST Action 231, 1999]. Es un modelo de bolsillo: solo necesita las funciones matemáticas básicas de MATLAB y el cálculo es casi instantáneo. La fórmula de pérdidas para entorno urbano medio ($C_m = 3$ dB) es:

$$L_{50} [\text{dB}] = 46,3 + 33,9 \log_{10} f_c - 13,82 \log_{10} h_{Tx} - a(h_{Rx}) + (44,9 - 6,55 \log_{10} h_{Tx}) \log_{10} d + C_m \quad (4.3)$$

con f_c en MHz, d en km y $a(h_{Rx})$ el factor de corrección de la antena receptora:

$$a(h_{Rx}) = (1,1 \log_{10} f_c - 0,7) h_{Rx} - (1,56 \log_{10} f_c - 0,8) \quad (4.4)$$

De ahí, $P_{rx} = \text{EIRP} + G_{rx} - L_{50}$.

El problema es que COST-231 Hata fue calibrado para 1500–2000 MHz. A 3,5 GHz se empieza a quedar corto: según la literatura introduce entre 2 y 4 dB de error extra respecto a medidas reales, lo que en términos de cobertura puede suponer 100–200 m de diferencia en el borde de celda. Por eso lo usamos solo como respaldo, no como resultado.

FSPL como referencia teórica

Sea cual sea el modelo activo, siempre calculamos también el FSPL (*Free Space Path Loss*) como cota superior teórica:

$$\text{FSPL} [\text{dB}] = 20 \log_{10} d + 20 \log_{10} f_c + 20 \log_{10} \left(\frac{4\pi}{c} \right) \quad (4.5)$$

donde d está en metros y f_c en Hz. La diferencia entre el FSPL y el modelo activo cuantifica directamente el efecto del terreno y del entorno sobre la propagación. Esta comparación la visualizamos en el bloque 8 y sirve de base para la figura de la el Capítulo 5.

4.1.6. Cálculo del RSRP (Bloque 5)

P_{rx} representa la potencia total integrada sobre todo el ancho de banda del canal, pero lo que miden los terminales 5G NR es el RSRP (*Reference Signal Received Power*): la potencia promedio en los *Resource Elements* que transportan el *Synchronization*

Signal Block, según 3GPP TS 38.215. Para pasar de una magnitud a la otra hay que normalizar por el número de RE del canal:

$$\text{RSRP [dBm]} = P_{rx} \text{ [dBm]} - 10 \log_{10}(N_{RB} \cdot 12) \quad (4.6)$$

donde el factor 12 es el número de subportadoras por *Resource Block*. Para n78 con 100 MHz de ancho de banda y SCS de 30 kHz, 3GPP TS 38.104 fija $N_{RB} = 66$, así que la corrección vale:

$$\Delta_{\text{RSRP}} = 10 \log_{10}(66 \times 12) = 10 \log_{10}(792) \approx 29,8 \text{ dB} \quad (4.7)$$

Los valores resultantes se clamean al rango $[-140, -40]$ dBm y se reconstruyen en matriz 60×60 para visualización y exportación. En el Cuadro 4.4 recogemos los umbrales de clasificación que usamos en el *heatmap*, extraídos de los criterios de KPI de 3GPP para 5G NR.

Listing 4.5: Cálculo del RSRP a partir de P_{rx} (Bloque 5).

```

1 NRB_100MHz = 66;           % Resource Blocks para 100 MHz, SCS=30 kHz
2 delta_RSRP_dB = 10 * log10(NRB_100MHz * 12); % aprox 29.0 dB
3
4 RSRP_dBm = Prx_dBm - delta_RSRP_dB;
5
6 RSRP_min = -140; RSRP_max = -40;           % Lmites fsicos
7 RSRP_dBm = max(min(RSRP_dBm, RSRP_max), RSRP_min);
8
9 RSRP_grid = reshape(RSRP_dBm, nPuntos, nPuntos);

```

Tabla 4.4: Clasificación de la cobertura en función del RSRP (3GPP).

Categoría	Rango RSRP	Descripción
Excelente	≥ -80 dBm	Muy buena calidad, velocidades máximas
Buena	$[-90, -80)$ dBm	Calidad adecuada para todos los servicios
Aceptable	$[-100, -90)$ dBm	Cobertura funcional, velocidades reducidas
Débil	$[-110, -100)$ dBm	Cobertura marginal, posibles cortes
Sin cobertura	< -110 dBm	Señal insuficiente para establecer conexión

4.1.7. Exportación de resultados a JSON (Bloque 7)

Con el RSRP calculado, el bloque 7 llama a `exportar_json.m`. La función genera un UUID v4 vía la JVM integrada en MATLAB (`java.util.UUID.randomUUID()`), empaqueta los metadatos y los puntos de cobertura en una estructura, serializa todo con `jsonencode` y escribe el fichero en `matlab/output/<uuid>.json`. No hay dependencias externas: la JVM viene de serie con cualquier instalación de MATLAB.

El formato de salida lo diseñamos para que el backend Spring Boot pueda leerlo sin transformaciones intermedias:

Listing 4.6: Estructura del fichero JSON generado por `exportar_json.m`.

```

1 {
2   "id": "a3f7c2b1-...",
3   "name": "URJC_Fuenlabrada_3.5GHz_40dBm",
4   "createdAt": "2026-04-15T12:00:00Z",
5   "metadata": {
6     "frequency_ghz": 3.5,
7     "tx_power_dbm": 40,
8     "antenna_height_m": 30,
9     "tx_location": { "latitude": 40.2897, "longitude": -3.8244 },
10    "model": "Longley-Rice",
11    "terrain_data": "SRTM_30m"
12  },
13  "coveragePoints": [
14    { "lat": 40.2764, "lng": -3.8356, "rsrp": -97.4 },
15    ...
16  ]
17 }
```

El array `coveragePoints` solo incluye los puntos con RSRP superior a -130 dBm; los puntos por debajo de ese umbral no aportan información útil al *heatmap* y engordan el fichero innecesariamente. Para la cuadrícula de 3600 puntos del escenario URJC, el JSON resultante ocupa entre 300 y 500 KB según cuántos puntos queden fuera del umbral.

El Listado 4.7 muestra el núcleo de la función `exportar_json.m`: la generación del UUID, la construcción de la estructura y la serialización. La función valida las entradas, genera la marca de tiempo en formato ISO 8601 con zona horaria UTC y escribe el fichero en la carpeta `matlab/output/`.

Listing 4.7: Fragmento principal de `exportar_json.m`.

```

1 function fichero_json = exportar_json(nombre, lats, lons, rsrp_dBm, params)
2   rsrp_dBm = max(min(double(rsrp_dBm), -40), -140); % Clamp
3   uuid     = char(java.util.UUID.randomUUID().toString());
4   ahora    = datetime('now', 'TimeZone', 'UTC');
5   createdAt = char(ahora, "yyyy-MM-dd'T'HH:mm:ss'Z'");
6
7   metadata.frequency_ghz = double(params.frecuencia_GHz);
8   metadata.tx_power_dbm  = double(params.potencia_tx_dBm);
9   metadata.antenna_height_m = double(params.altura_tx_m);
10  metadata.tx_location     = struct('latitude', params.lat_emplazamiento,
11    ...
12    'longitude', params.lon_emplazamiento);
13  metadata.model           = char(string(params.modelo_propagacion));
14  metadata.terrain_data    = char(string(params.datos_elevacion));
```

```

14
15     idx_validos = rsrp_dBm >= -130; % Filtrar puntos sin seal til
16     coveragePoints = struct('lat', num2cell(lats(idx_validos)), ...
17                             'lng', num2cell(lons(idx_validos)), ...
18                             'rsrp', num2cell(rsrp_dBm(idx_validos)));
19
20     sim_struct = struct('id', uuid, 'name', nombre, ...
21                         'createdAt', createdAt, 'metadata', metadata, ...
22                         'coveragePoints', coveragePoints);
23
24     json_str = jsonencode(sim_struct, 'PrettyPrint', true);
25     fichero_json = fullfile(output_dir, [uuid, '.json']);
26     fid = fopen(fichero_json, 'w', 'n', 'UTF-8');
27     fprintf(fid, '%s', json_str); fclose(fid);
28 end

```

4.2. Backend: API REST con Spring Boot

El backend tiene una única responsabilidad: exponer los ficheros JSON generados por MATLAB como recursos accesibles vía HTTP para que el frontend pueda consumirlos. Para ello se diseñó una API REST sin estado (*stateless*) con Spring Boot 3.2.4 y Java 17, empaquetada como un JAR ejecutable que no requiere servidor de aplicaciones externo.

4.2.1. Estructura del proyecto

El proyecto sigue la estructura canónica de Maven con Spring Boot. El fichero `pom.xml` declara tres dependencias principales: `spring-boot-starter-web` (servidor embebido Tomcat, Jackson para JSON y el framework web de Spring MVC), `springdoc-openapi-starter-webmvc-ui` 2.3.0 (generación automática de la documentación OpenAPI 3 y la interfaz Swagger UI) y `spring-boot-starter-test` (JUnit 5, Mockito y utilidades de test de Spring).

El código Java se organiza en cinco paquetes siguiendo el patrón clásico de capas:

Tabla 4.5: Estructura de paquetes del backend.

Paquete	Responsabilidad
<code>controller</code>	Controlador REST: define endpoints y formato de respuesta
<code>service</code>	Lógica de negocio: lectura y validación de ficheros JSON
<code>model</code>	DTOs: <code>SimulationSummaryDto</code> (record Java)
<code>exception</code>	Excepciones de dominio y manejador global
<code>config</code>	Configuración CORS para desarrollo local

La clase de arranque `Gemelo5gApplication` es un `@SpringBootApplication`

estándar que delega en `SpringApplication.run()` el escaneo de componentes y la configuración automática del contenedor.

4.2.2. Servicio de simulaciones

`SimulationService` es el componente central del backend. Anotado con `@Service`, recibe por inyección de dependencias la ruta del directorio de datos (propiedad `simulations.data-dir` de `application.properties`) y un `ObjectMapper` de Jackson para la deserialización JSON.

El directorio de datos se normaliza en el constructor con `Paths.get(path).toAbsolutePath().normalize()` para prevenir ataques de *path traversal*: cualquier ruta resuelta que no empiece por el directorio base se rechaza con un `SimulationNotFoundException`. Además, antes de acceder al sistema de ficheros, el método `getId` valida que el identificador recibido sea un UUID v4 conforme a RFC 4122 mediante la expresión regular:

`[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}`

Si el identificador no supera ninguna de las dos validaciones, se lanza la misma excepción para no revelar información sobre la estructura del sistema de ficheros.

Listing 4.8: Métodos principales de `SimulationService`.

```

1  @Service
2  public class SimulationService {
3      private final Path dataDir;
4      private final ObjectMapper mapper;
5
6      public SimulationService(
7          @Value("${simulations.data-dir}") String dataDirPath,
8          ObjectMapper mapper) {
9          this.dataDir = Paths.get(dataDirPath).toAbsolutePath().normalize();
10         this.mapper = mapper;
11     }
12
13     public List<SimulationSummaryDto> listAll() throws IOException {
14         if (!Files.isDirectory(dataDir)) return List.of();
15         try (Stream<Path> files = Files.list(dataDir)) {
16             return files.filter(p -> p.toString().endsWith(".json"))
17                 .map(this::parseSummary).toList();
18         }
19     }
20
21     private static final String UUID_REGEX =
22         "[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}"

```

```

23     + "-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}";
24
25     public JsonNode getById(String id) throws IOException {
26         if (!id.matches(UUID_REGEX))
27             throw new SimulationNotFoundException(id);
28         Path file = dataDir.resolve(id + ".json").normalize();
29         if (!file.startsWith(dataDir) || !Files.exists(file))
30             throw new SimulationNotFoundException(id);
31         return mapper.readTree(file.toFile());
32     }
33 }

```

El método `listAll` recorre el directorio con `Files.list()`, filtra los ficheros `.json` y mapea cada uno a un `SimulationSummaryDto` que contiene solo los campos ligeros (`id`, `name`, `createdAt`, `metadata`), sin incluir el array `coveragePoints`. De este modo, el listado no satura al cliente cuando hay múltiples simulaciones almacenadas.

4.2.3. Controlador REST

`SimulationController` expone tres endpoints bajo el prefijo `/api/simulations`, todos de solo lectura (GET). El Cuadro 4.6 los resume.

Tabla 4.6: Endpoints de la API REST.

Método	URL	Descripción
GET	<code>/api/simulations</code>	Listado de simulaciones (sin puntos)
GET	<code>/api/simulations/{id}</code>	JSON completo con <code>coveragePoints</code>
GET	<code>/api/simulations/{id}/export</code>	Descarga como fichero adjunto

El endpoint de listado soporta paginación opcional: si el cliente envía los parámetros `?page=0&size=10`, el controlador calcula los índices de corte sobre la lista completa y devuelve un DTO `PageResponse` con los campos `content`, `totalElements`, `page`, `size` y `totalPages`. Si no se envía el parámetro `page`, devuelve la lista completa para mantener la compatibilidad con clientes simples.

Listing 4.9: Endpoints del controlador REST.

```

1 @RestController
2 @RequestMapping("/api/simulations")
3 public class SimulationController {
4     private final SimulationService service;
5
6     @GetMapping(produces = MediaType.APPLICATION_JSON_VALUE)
7     public ResponseEntity<?> listSimulations(
8         @RequestParam(required = false) Integer page,
9         @RequestParam(required = false, defaultValue = "10") int size)
10    throws IOException {

```

```

11     List<SimulationSummaryDto> all = service.listAll();
12     if (page == null) return ResponseEntity.ok(all);
13     int total = all.size();
14     int from = Math.min(page * size, total);
15     int to = Math.min(from + size, total);
16     return ResponseEntity.ok(
17         new PageResponse<>(all.subList(from, to), total, page, size,
18             (int) Math.ceil((double) total / size)));
19 }
20
21 @GetMapping(value =("/{id}", produces = MediaType.APPLICATION_JSON_VALUE)
22 public JsonNode getSimulation(@PathVariable String id) throws
23     IOException {
24     return service.getId(id);
25 }
26
27 @GetMapping(value =("/{id}/export", produces =
28     MediaType.APPLICATION_JSON_VALUE)
29 public ResponseEntity<Resource> exportSimulation(@PathVariable String id)
30     throws IOException {
31     Resource resource = service.getFileResource(id);
32     return ResponseEntity.ok()
33         .contentType(MediaType.APPLICATION_JSON)
34         .header(HttpHeaders.CONTENT_DISPOSITION,
35             "attachment; filename=\"" + id + ".json\"")
36         .body(resource);
37 }

```

El endpoint de exportación añade la cabecera `Content-Disposition: attachment` para que el navegador descargue el JSON como fichero en vez de mostrarlo en línea. Internamente delega en `SimulationService.getFileResource()`, que devuelve un `FileSystemResource` apuntando al fichero original.

4.2.4. Gestión de errores

La gestión centralizada de errores se apoya en dos clases. Primero, `SimulationNotFoundException` extiende `RuntimeException` y lleva la anotación `@ResponseStatus(HttpStatus.NOT_FOUND)`: cuando se lanza, Spring devuelve automáticamente un código 404. Segundo, `GlobalExceptionHandler` (anotado con `@RestControllerAdvice`) captura las excepciones no manejadas y las transforma en respuestas JSON consistentes con estructura `{ "error": "...", "detail": "..." }`:

Listing 4.10: Manejador global de excepciones.

```

1 @RestControllerAdvice
2 public class GlobalExceptionHandler {
3     @ExceptionHandler(SimulationNotFoundException.class)

```

```

4      @ResponseStatus(HttpStatus.NOT_FOUND)
5      public Map<String, String> handleNotFound(SimulationNotFoundException
        ex) {
6          return Map.of("error", "Simulacion no encontrada",
7                        "detail", ex.getMessage());
8      }
9
10     @ExceptionHandler(IOException.class)
11     @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
12     public Map<String, String> handleIOException(IOException ex) {
13         return Map.of("error", "Error leyendo datos de simulacion",
14                       "detail", ex.getMessage());
15     }
16 }

```

Este diseño garantiza que el frontend siempre recibe un JSON interpretable, nunca una traza de pila ni una página de error HTML por defecto de Spring.

4.2.5. Configuración y CORS

El fichero `application.properties` contiene tres grupos de configuración:

Listing 4.11: Configuración del backend (`application.properties`).

```

1 server.port=8080
2 simulations.data-dir=../matlab/output
3 springdoc.swagger-ui.path=/swagger-ui.html
4 springdoc.api-docs.path=/api-docs

```

La propiedad `simulations.data-dir` apunta por defecto al directorio `matlab/output` relativo a la carpeta del backend, de modo que el servidor lee directamente los JSON que genera MATLAB sin copia intermedia. En producción o en entornos con rutas distintas, basta con cambiar esta propiedad o pasarla como variable de entorno (`SIMULATIONS_DATA_DIR`).

La clase `CorsConfig` registra una política CORS que permite peticiones desde `http://localhost:4200` (el servidor de desarrollo de Angular) con los métodos `GET` y `OPTIONS`. Esta configuración es imprescindible durante el desarrollo local, ya que navegador y backend corren en puertos distintos y sin la cabecera `Access-Control-Allow-Origin` el navegador bloquearía las peticiones.

Finalmente, `springdoc-openapi` genera automáticamente la especificación OpenAPI 3 a partir de las anotaciones `@Operation` y `@Tag` del controlador. La interfaz Swagger UI queda accesible en `http://localhost:8080/swagger-ui.html` y permite probar los endpoints directamente desde el navegador sin necesidad de herramientas externas como Postman.

4.2.6. Tests unitarios

El servicio de simulaciones cuenta con once tests unitarios escritos con JUnit 5. La anotación `@TempDir` de JUnit crea un directorio temporal limpio antes de cada test, lo que garantiza el aislamiento entre ejecuciones. Los casos de prueba se agrupan en tres categorías:

Tabla 4.7: Resumen de los tests unitarios del backend.

Método	Caso	Resultado esperado
<code>getIdById</code>	UUID válido con fichero existente	Devuelve <code>JsonNode</code> correcto
<code>getIdById</code>	UUID válido pero fichero inexistente	<code>SimulationNotFoundException</code>
<code>getIdById</code>	Formato de UUID no válido	<code>SimulationNotFoundException</code>
<code>getIdById</code>	Intento de <i>path traversal</i>	<code>SimulationNotFoundException</code>
<code>listAll</code>	Directorio vacío	Lista vacía
<code>listAll</code>	Directorio con JSON y no-JSON	Solo devuelve los <code>.json</code>
<code>getFileResource</code>	UUID válido	Recurso legible
<code>getFileResource</code>	Formato inválido	<code>SimulationNotFoundException</code>

Los tests de *path traversal* verifican que cadenas como `../.../etc/passwd` o prefijos con secuencias codificadas (`%2F`) son rechazadas antes de tocar el sistema de ficheros, validando así la defensa en profundidad implementada en `SimulationService`.

4.3. Frontend: Aplicación Angular

La capa de presentación es una aplicación web monopágina (*Single Page Application*) construida con Angular 17 y TypeScript 5.9. Su función es consumir la API REST del backend y representar los datos de cobertura como un *heatmap* interactivo sobre un mapa geográfico. A diferencia de versiones anteriores de Angular, la aplicación utiliza exclusivamente componentes *standalone* (sin `NgModule`) y gestión de estado basada en *signals*, siguiendo las prácticas recomendadas a partir de Angular 16.

4.3.1. Arquitectura de componentes

La aplicación está compuesta por siete componentes, cada uno independiente y declarado con `standalone: true`. El Cuadro 4.9 los enumera junto con su responsabilidad.

Tabla 4.9: Componentes de la aplicación Angular.

Componente	Responsabilidad
HeaderComponent	Barra superior con título y toggle de tema
SidebarComponent	Selector de simulación, filtro RSRP, estadísticas
MapComponent	Mapa Leaflet con <i>heatmap</i> y marcador TX
InfoPanelComponent	Detalle del punto seleccionado al hacer clic
StatsPanelComponent	Resumen estadístico de la simulación activa
CoverageLegendComponent	Leyenda de colores del <i>heatmap</i>
SimulationConfigComponent	Parámetros de configuración de la simulación

`AppComponent` actúa como raíz: importa los componentes `Header`, `Sidebar`, `Map` e `InfoPanel`, y en su método `ngOnInit` invoca `SimulationService.loadSimulations()` para cargar el listado inicial desde el backend. Si la petición falla, se muestra un *banner* de error con botón de reintento que vuelve a llamar al mismo método.

4.3.2. Servicio de simulaciones

`SimulationService` centraliza toda la comunicación con la API y el estado de la aplicación. Está registrado con `providedIn: 'root'`, lo que lo convierte en un *singleton* inyectable en cualquier componente.

El estado interno se gestiona con cinco *signals* de escritura (`WritableSignal`): `simulations` (lista cargada), `selectedSimulationId` (UUID activo), `rsrpThreshold` (umbral del filtro, inicializado a -140 dBm), `selectedPoint` (punto clicado en el mapa), `loading` y `error`. A partir de ellos se derivan cuatro *computed signals* de solo lectura, entre los que destaca `filteredCoveragePoints`:

Listing 4.12: Servicio de simulaciones (fragmento de `simulation.service.ts`).

```

1 @Injectable({ providedIn: 'root' })
2 export class SimulationService {
3   private readonly http = inject(HttpClient);
4   private simulations      = signal<Simulation[]>([]);
5   private selectedSimulationId = signal<string | null>(null);
6   private rsrpThreshold     = signal<number>(-140);
7   readonly loading          = signal<boolean>(false);
8   readonly error            = signal<string | null>(null);
9
10  readonly filteredCoveragePoints = computed(() => {
11    const sim = this.currentSimulation();
12    const thr = this.rsrpThreshold();
13    if (!sim) return [];
14    return sim.coverage_points.filter(p => p.rsrp_dbm >= thr);
15  });
16

```

```

17  async loadSimulations(): Promise<void> {
18      this.loading.set(true); this.error.set(null);
19      try {
20          const summaries = await firstValueFrom(
21              this.http.get<ApiSimulationSummary[]>(API_BASE));
22          const partial: Simulation[] = summaries.map(s => ({
23              simulation_id: s.id, metadata: s.metadata,
24              coverage_area: { width_km: 3, height_km: 3, grid_spacing_m: 50 },
25              coverage_points: []
26          }));
27          this.simulations.set(partial);
28          if (partial.length > 0)
29              await this.selectSimulation(partial[0].simulation_id);
30      } catch (e) {
31          this.error.set('No se pudo conectar con el backend.');
```

El flujo de carga opera en dos fases: primero se solicitan los resúmenes (sin `coveragePoints`) para rellenar el selector del *sidebar*; al seleccionar una simulación, se pide el JSON completo con los puntos de cobertura y se actualiza la lista reactiva con `simulations.update()`. Este patrón evita transferir datos pesados hasta que el usuario los necesita realmente.

La función `normalizeRsrp` convierte el valor RSRP al rango $[0, 1]$ para el gradiente del *heatmap* mediante la expresión $(\text{RSRP} + 140)/100$, donde -140 dBm se mapea a 0 y -40 dBm a 1.

4.3.3. Componente de mapa

`MapComponent` es el componente más complejo de la interfaz. Integra la librería Leaflet 1.9.4 con el *plugin* `leaflet.heat` para renderizar el mapa de calor de cobertura sobre tiles cartográficos de CARTO (*dark_all*).

El mapa se inicializa en `ngOnInit` centrado en las coordenadas del campus URJC ($40,2897^\circ$ N, $3,8244^\circ$ W) con zoom 13. Un `effect()` de Angular observa los cambios en `filteredCoveragePoints` y en `currentSimulation`: cada vez que alguno de esos *signals* cambia, se reconstruye el *heatmap* automáticamente.

Listing 4.13: Renderizado del *heatmap* de cobertura.

```

1  private updateHeatmap(points: CoveragePoint[]): void {
2      if (this.heatmapLayer) {
3          this.map.removeLayer(this.heatmapLayer);
4          this.heatmapLayer = null;
5      }
6      if (!this.heatmapVisible || points.length === 0) return;
```

```

7
8   const heatData: [number, number, number][] = points.map(p => [
9     p.latitude,
10    p.longitude,
11    this.simulationService.normalizeRsrp(p.rsrp_dbm) // (rsrp+140)/100
12  ]);
13
14  this.heatmapLayer = (L as any).heatLayer(heatData, {
15    radius: 18, blur: 15, maxZoom: 17, max: 1.0,
16    gradient: {
17      0.0: '#ef4444', // Debil    (< -100 dBm)
18      0.4: '#f97316', // Aceptable(-100..-85)
19      0.55: '#fbbf24', // Bueno    (-85..-70)
20      0.7: '#10b981'  // Excelente (> -70 dBm)
21    }
22  });
23  this.heatmapLayer.addTo(this.map);
24 }

```

El gradiente de colores replica los umbrales de clasificación del Cuadro 4.4: rojo para señal débil, naranja para aceptable, amarillo para buena y verde para excelente. `leaflet.heat` dibuja todos los puntos en un único elemento `<canvas>`, lo que permite renderizar los 3600 puntos de la cuadrícula sin penalización visible de rendimiento.

El manejador de *click* del mapa busca el punto de cobertura más cercano al lugar pulsado mediante distancia euclidiana en grados. Si la distancia es inferior a $0,003^\circ$ (aproximadamente 300 m), selecciona el punto y el `InfoPanelComponent` muestra su RSRP, coordenadas y *path loss* estimado. En caso contrario, se deselecciona cualquier punto previo.

4.3.4. Interfaz de usuario

La aplicación ofrece un cambio de tema claro/oscuro gestionado por `ThemeService`. Este servicio mantiene un *signal* booleano (`darkMode`), persiste la preferencia en `localStorage` con la clave `gd5g-theme` y aplica la clase CSS `theme-light` al `<body>` cuando el usuario alterna al modo claro. El tema oscuro es el predeterminado, ya que los mapas con tiles CARTO *dark_all* se integran mejor visualmente con fondos oscuros.

El *sidebar* izquierdo ofrece tres controles principales: un selector `<select>` que lista las simulaciones disponibles (por nombre de escenario o UUID), un deslizador (*slider*) de rango $[-140, -40]$ dBm que filtra en tiempo real los puntos visibles en el *heatmap*, y un contador que muestra cuántos puntos pasan el filtro respecto al total.

El `InfoPanelComponent` se activa al hacer clic en un punto del mapa y muestra el nivel RSRP con un indicador visual de barras (1 a 4 barras activas según la calidad),

las coordenadas geográficas exactas, el rango de umbrales 3GPP y una estimación del *path loss*: $PL \approx \text{EIRP} - \text{RSRP}$, donde EIRP se aproxima como $P_{tx} + 13$ dB a partir de los metadatos de la simulación.

4.3.5. Gestión del estado con *signals*

Angular introdujo los *signals* en la versión 16 como alternativa a la gestión de estado basada en `Observable` de RxJS o librerías externas como NgRx. En esta aplicación hemos adoptado *signals* de forma integral por tres razones:

1. **Simplicidad:** un `WritableSignal<T>` se lee como una función (`signal()`) y se escribe con `signal.set()` o `signal.update()`, sin necesidad de suscripciones ni operadores.
2. **Derivación reactiva:** `computed()` crea valores derivados que Angular recalcula automáticamente cuando sus dependencias cambian. En nuestro caso, `filteredCoveragePoints` depende de `currentSimulation` y de `rsrpThreshold`, y se actualiza sin intervención manual.
3. **Efectos secundarios controlados:** `effect()` ejecuta una función cada vez que los *signals* leídos dentro de ella cambian. Lo usamos en `MapComponent` para reconstruir el *heatmap* cuando cambian los datos filtrados.

El modelo de datos del frontend se define en `simulation.model.ts` mediante interfaces TypeScript: `Simulation`, `CoveragePoint`, `SimulationMetadata` y `SimulationStats`. La función `apiToSimulation()` se encarga de mapear el formato del backend (con campos `lat`, `lng`, `rsrp`) al formato interno (con campos `latitude`, `longitude`, `rsrp_dbm`), desacoplando así el modelo de la API del modelo de la vista.

4.4. Integración del sistema

Las tres capas descritas en las secciones anteriores forman un *pipeline* lineal de datos que no requiere base de datos ni colas de mensajes: la simulación genera ficheros, el backend los sirve y el frontend los renderiza. Esta sencillez es deliberada y responde a los requisitos del escenario táctico, donde el sistema debe poder desplegarse en un portátil de campo sin infraestructura adicional.

4.4.1. Flujo de datos extremo a extremo

El flujo completo consta de cuatro etapas:

1. **Simulación:** el operador ejecuta `simulacion.cobertura.5g.m` en MATLAB con los parámetros deseados (frecuencia, potencia, emplazamiento). El bloque 7 invoca `exportar_json.m`, que genera un fichero `<uuid>.json` en `matlab/output/`.
2. **Servicio:** Spring Boot, configurado con `simulations.data-dir=../matlab/output`, detecta los ficheros JSON del directorio y los expone en la API REST sin necesidad de reiniciar ni recargar.
3. **Presentación:** la aplicación Angular solicita el listado de simulaciones al arrancar (`GET /api/simulations`), presenta el selector en el *sidebar* y, cuando el usuario elige una, solicita el JSON completo (`GET /api/simulations/{uuid}`).
4. **Visualización:** con los `coveragePoints` recibidos, el `MapComponent` construye el *heatmap* sobre Leaflet y lo muestra en pantalla.

Este diseño desacoplado permite que cada capa evolucione de forma independiente. Basta con respetar el contrato JSON para que un cambio en el modelo de propagación de MATLAB no afecte ni al backend ni al frontend.

4.4.2. Formato de intercambio JSON

El fichero JSON constituye el contrato de interfaz entre MATLAB y Spring Boot. Su esquema, definido implícitamente por `exportar_json.m` y consumido por `SimulationService` vía Jackson, contiene cinco campos raíz:

Tabla 4.10: Esquema del fichero JSON de simulación.

Campo	Tipo	Descripción
<code>id</code>	<code>string</code>	UUID v4 generado por la JVM de MATLAB
<code>name</code>	<code>string</code>	Nombre descriptivo del escenario
<code>createdAt</code>	<code>string</code>	Marca temporal ISO 8601 (UTC)
<code>metadata</code>	<code>object</code>	Parámetros normalizados (frecuencia, modelo, etc.)
<code>parameters</code>	<code>object</code>	Copia de la estructura <code>params</code> de MATLAB
<code>coveragePoints</code>	<code>array</code>	Array de <code>{lat, lng, rsrp}</code>

El campo `metadata` expone los parámetros en claves normalizadas en inglés (`frequency_ghz`, `tx_power_dbm`, `antenna_height_m`, `tx_location`, `model`, `terrain_data`) para que el frontend pueda consumirlos directamente sin conocer la nomenclatura interna de MATLAB. El campo `parameters` conserva los nombres originales de la estructura MATLAB por compatibilidad con análisis posteriores.

En el lado del backend, la deserialización la realiza Jackson a través de `ObjectMapper.readTree()`, que construye un árbol `JsonNode` genérico. Para el listado, `parseSummary` extrae solo `id`, `name`, `createdAt` y `metadata` del nodo raíz, evitando cargar en memoria el array `coveragePoints` que puede contener miles de elementos.

En el lado del frontend, la función `apiToSimulation()` (definida en `simulation.model.ts`) mapea los campos del JSON a la interfaz `Simulation`, renombrando `lat/lng/rsrp` a `latitude/longitude/rsrp_dbm` para mantener la coherencia con el modelo interno de la aplicación.

4.4.3. Configuración del entorno de desarrollo

Para ejecutar el sistema completo en un entorno de desarrollo local se necesitan los siguientes prerequisites:

Tabla 4.11: Prerrequisitos del entorno de desarrollo.

Herramienta	Versión mínima	Propósito
MATLAB	R2024a	Simulación radioeléctrica
Java JDK	17	Compilación y ejecución del backend
Apache Maven	3.9+	Gestión de dependencias del backend
Node.js	18 LTS	Entorno de ejecución del frontend
Angular CLI	17+	Compilación y servidor de desarrollo

El orden de arranque recomendado es:

1. Ejecutar la simulación en MATLAB para generar al menos un fichero JSON en `matlab/output/`.
2. Arrancar el backend desde la carpeta `backend/` con `mvn spring-boot:run`. El servidor queda disponible en `http://localhost:8080`.
3. Arrancar el frontend desde `mockups/mockup_v1/gemelo-digital-5g/` con `ng serve`. La aplicación queda disponible en `http://localhost:4200`.

La configuración CORS del backend (sección 4.2.5) permite que el frontend en el puerto 4200 realice peticiones al backend en el puerto 8080 sin que el navegador las bloquee. En un despliegue unificado (por ejemplo, sirviendo los *assets* estáticos de Angular desde el propio Spring Boot), esta configuración dejaría de ser necesaria.

4.4.4. Despliegue

El backend se empaqueta como un JAR ejecutable estándar de Spring Boot:

Listing 4.14: Compilación y ejecución del backend.

```
1 cd backend/  
2 mvn clean package -DskipTests  
3 java -jar target/gemelo5g-0.1.0-SNAPSHOT.jar \  
4     --simulations.data-dir=/ruta/a/jsons
```

El comando `mvn package` compila el código, ejecuta los tests (omitidos aquí con `-DskipTests` para acelerar) y produce un JAR con todas las dependencias incluidas gracias al *plugin* `spring-boot-maven-plugin`. La propiedad `simulations.data-dir` se puede sobrescribir por línea de comandos para apuntar a cualquier directorio que contenga los JSON de simulación.

Para el frontend, `ng build` genera los *assets* estáticos en `dist/`, que pueden servirse con cualquier servidor HTTP (Nginx, Apache, o el propio Spring Boot copiándolos a `src/main/resources/static/`).

En el contexto de este TFG, ambos servicios se ejecutan en la misma máquina durante el desarrollo y las pruebas. Un despliegue táctico real seguiría el mismo esquema en un equipo portátil de campo, donde la latencia de red es irrelevante y la prioridad es la autonomía del sistema sin dependencias de conectividad externa.

Capítulo 5

Validación y Resultados

5.1. Diseño del Escenario de Prueba

Campus URJC Fuenlabrada: 40.2897°N, 3.8244°W.

Parámetros.

5.2. Resultados de la Simulación MATLAB

Alcance.

Variación RSRP.

RSRP medio en cobertura.

Validación teórica.

5.3. Resultados de la Integración Web

Métricas rendimiento.

Compatibilidad.

Cumplimiento RNF2.

5.4. Comparación con Software Comercial

Tabla comparativa Xirio Online vs TFG.

Conclusión.

5.5. Limitaciones Identificadas

Modelo propagación: no obstáculos 3D, limitado en urbano denso, no caso indoor.

Implementación: simulación offline, no optimizado para número de usuarios, sin persistencia DB, sin autenticación.

Capítulo 6

Conclusiones y Trabajo Futuro

6.1. Conclusiones Generales

PoC exitoso demuestra viabilidad de gemelo digital web para 5G táctico con tecnologías modernas gratuitas.

Arquitectura tres capas adecuada para validación preliminar rápida.

6.2. Consecución de Objetivos

OE1-OE5: todos conseguidos?? (modelado, interoperabilidad, backend, frontend, validación).

6.3. Competencias Adquiridas

Propagación radioeléctrica, desarrollo fullstack, GIS, gestión proyectos ágil, investigación técnica.

6.4. Aplicabilidad Práctica

Emergencias, eventos temporales, zonas remotas, formación universitaria.

6.5. Limitaciones del Trabajo

Modelo simplificado, simulación offline, sin optimización ubicación, interfaz básica, sin telemetría real.

6.6. Líneas de Trabajo Futuro

Mejoras modelo: ray-tracing, obstáculos 3D, multi-frecuencia, indoor.

Arquitectura tiempo real: GPU, WebSocket streaming, cloud distribuido, caché Redis.

Funcionalidades: optimización automática TX (algoritmo genético), multi-antena,

análisis interferencia, estimación capacidad.

Integración externa: equipos reales, telemetría vivo, GIS profesional (QGIS).

Usabilidad: PWA offline, app móvil nativa, colaboración multi-usuario.

6.7. Impacto y Difusión

Publicación IEEE, repositorio GitHub open-source, transferencia empresas, uso educativo URJC.

6.8. Reflexión Personal

Apéndice A

Anexo A: Manual de Usuario

Uso interfaz web, interpretación resultados.

Apéndice B

Anexo B: Manual de Instalación

Setup MATLAB, JDK, Node.js, IDEs, scripts build.

Apéndice C

Anexo C: Código Fuente Destacado

Fragmentos MATLAB, Java, TypeScript comentados.

Apéndice D

Anexo D: Especificación API REST

OpenAPI 3.0, esquemas JSON, ejemplos cURL.

Apéndice E

Anexo E: Resultados Adicionales

Escenarios rural montañoso y urbano denso.

Apéndice F

Anexo F: Diagramas UML

Clases, secuencia, componentes, despliegue.

Bibliografía

- [3GPP, 2018] 3GPP (2018). Technical specification group radio access network; nr; summary of release 15 work items. Technical Report TR 21.915 v15.0.0, 3rd Generation Partnership Project.
- [3GPP, 2020] 3GPP (2020). Technical specification group radio access network; summary of release 16 work items. Technical Report TR 21.916 v1.5.0, 3rd Generation Partnership Project.
- [3GPP, 2022a] 3GPP (2022a). Technical specification group radio access network; nr; base station (bs) radio transmission and reception. Technical Report TS 38.104 v17.5.0, 3rd Generation Partnership Project.
- [3GPP, 2022b] 3GPP (2022b). Technical specification group radio access network; nr; physical layer measurements. Technical Report TS 38.215 v17.2.0, 3rd Generation Partnership Project.
- [3GPP, 2022c] 3GPP (2022c). Technical specification group radio access network; study on channel model for frequencies from 0.5 to 100 ghz. Technical Report TR 38.901 v17.0.0, 3rd Generation Partnership Project.
- [3GPP, 2022d] 3GPP (2022d). Technical specification group radio access network; summary of release 17 work items. Technical Report TR 21.917 v17.0.0, 3rd Generation Partnership Project.
- [Alraih et al., 2022] Alraih, S., Shayea, I., Behjati, M., Nordin, R., Abdullah, N. F., Abu-Samah, A., and Nandi, D. (2022). Revolution or evolution? technical requirements and considerations towards 6G mobile communications. *Sensors*, 22(3):762.
- [Camp and Knightly, 2020] Camp, J. D. and Knightly, E. W. (2020). Wireless ephemeral networks: Enabling highly mobile tactical 5G cells. *IEEE Communications Magazine*, 58(11):54–60.

- [CloudRF, 2024] CloudRF (2024). CloudRF — radio coverage and link budget tool. <https://cloudrf.com>.
- [COST Action 231, 1999] COST Action 231 (1999). Digital mobile radio towards future generation systems: Final report. Technical Report EUR 18957, European Commission, Brussels, Belgium.
- [Fuller et al., 2020] Fuller, A., Fan, Z., Day, C., and Barlow, C. (2020). Digital twin: Enabling technologies, challenges and open research. *IEEE Access*, 8:108952–108971.
- [Gast, 2005] Gast, M. (2005). 802.11 wireless networks: The definitive guide. O'Reilly Media, 2^a edición.
- [Grieves, 2014] Grieves, M. (2014). Digital twin: Manufacturing excellence through virtual factory replication. In *White Paper, Florida Institute of Technology*. Originalmente presentado en 2002 en la Universidad de Michigan.
- [Hata, 1980] Hata, M. (1980). Empirical formula for propagation loss in land mobile radio services. *IEEE Transactions on Vehicular Technology*, 29(3):317–325.
- [Hufford et al., 1982] Hufford, G. A., Longley, A. G., and Kissick, W. A. (1982). A guide to the use of the ITS irregular terrain model in the area prediction mode. Technical Report NTIA Report 82-100, Institute for Telecommunication Sciences, NTIA, Boulder, Colorado, USA.
- [INDRA / Ingeniería de Sistemas para la Defensa de España, 2024] INDRA / Ingeniería de Sistemas para la Defensa de España (2024). Xirio online — herramienta de planificación radioeléctrica. <https://xirio-online.com>.
- [ITU-T, 2021] ITU-T (2021). Digital twin network: Concepts and challenges. Technical Report ITU-T FG-AN Output Document, International Telecommunication Union.
- [Longley and Rice, 1968] Longley, A. G. and Rice, P. L. (1968). Prediction of tropospheric radio transmission loss over irregular terrain: A computer method. Technical Report ERL 79-ITS 67, Institute for Telecommunication Sciences, ESSA, Boulder, Colorado, USA.
- [Magliacane, 2011] Magliacane, J. A. (2011). SPLAT! — RF signal propagation, loss, and terrain analysis tool. <https://www.qsl.net/kd2bd/splat.html>.
- [Nguyen et al., 2021] Nguyen, H. X., Trestian, R., To, D., and Tatipamula, M. (2021). Digital twin for 5G and beyond. *IEEE Communications Magazine*, 59(2):10–15.

- [Okumura et al., 1968] Okumura, Y., Ohmori, E., Kawano, T., and Fukuda, K. (1968). Field strength and its variability in VHF and UHF land-mobile radio service. *Review of the Electrical Communication Laboratory*, 16(9-10):825–873.